

Supplementary Material: When the Interval Is Smaller Than the Instrument

GUSTAVO PEDRO RICO, School of Computer Science and Statistics, Trinity College Dublin, Ireland

This document is supplementary material and is not part of the main submission. Main-text references of the form “supplementary material S_n ” resolve here. The provenance of every section — what moved here, from where, and when — is indexed in the artefact at docs/supplement_index.md.

1 S1: The replay-rate provenance gap, in full

Auditing the corpus turned the same scrutiny on our own record-keeping, and it did not survive it either. We report the finding here because the protocol in the main text’s protocol section is weaker without it.

Replay plans built from StatsBomb carry a **baked-in 120× time compression**: a plan’s emission offset is already the match time divided by 120. The producer’s `--speedup` flag then multiplies that. So `--speedup 1` against such a plan is *not* real time, it is 120×; true real time needs 1/120. The same flag means different things depending on which plan it is pointed at, and getting it wrong is silent: the run completes, the wall time looks unremarkable, and the numbers stay plausible. We lost a campaign to it before adding a pre-flight that derives the value from the plan and then checks the achieved rate against elapsed wall time.

The part we cannot repair is retrospective. **No surviving artefact records the achieved replay rate of any run we report.** Several superseded Testbed A scenarios record the nominal `--speedup flag`, which is not the same thing: converting a flag to a rate requires the plan’s compression, and every plan those files name has since been deleted. Everywhere else the orchestrator wrote speedup into a per-campaign summary that lived beside the raw per-run data, and for the earliest corpora only the aggregated CSVs were kept. For the E1 corpus of the main text’s powered-transport section the surviving evidence is contradictory: the commit that introduced it states the runs were made at true real time, the campaign script reconstructed afterwards specifies `--speedup 10`, and the flag’s meaning was corrected 21 hours later the same day. E1 was replayed at 1×, 120× or 1200×, and we cannot say which.

The rate is recoverable from the measurements, if not from the metadata. Having argued all paper that a physical constraint can decide what record-keeping cannot, we applied the same move here, and it works. The start-up cost of the main text’s attribution section is a clock: it lasts a fixed 102.6 ms of *wall* time, so the number of events it swallows depends directly on the replay rate. Counting how many events per run carry it therefore reads the rate back out of the data.

The E1 runs sort into cells by how many events they matched, and one cell is diagnostic. Fifteen Kafka runs matched exactly eight events, and their median scheduling lag is 52.34 ms (range 51.60–53.01) — not 103 ms, and not 1.6 ms. A median of eight values is the mean of the fourth and fifth. The only way to land midway between the two modes is for exactly four of the eight to be late, so the fourth value is a fast event and the fifth a slow one: $(1.59 + 103.5)/2 = 52.6$ ms against 52.34 observed. Four is also exactly what the loop trace shows at a verified real-time rate (the window table in the main text), and it is what the plan predicts, since five events share $t_{\text{sim}}=0$ and the first of them pays the block rather than inheriting it.

That cell excludes the accelerated alternatives outright. At 120× the prologue spans 12.3 s of match time and holds 16 events; at 1200×, 123 s and 112 events. Under either, all eight matched events would be inside it and the cell would

read 103 ms. It reads half that. **E1 was replayed at true real time**, the commit record was right, and the campaign script reconstructed afterwards describes a different, earlier campaign. The remaining cells agree: runs matching five or seven events read 103 ms, as four late events out of five or seven must give.

The inference has since been checked against the original script, and it holds. Archiving the cloud driver before releasing it turned up the campaign as actually invoked, in a shell script in the operator’s home directory that no part of the repository had ever referenced. It specifies `--speedup 0.008333` — $1/120$, against a plan already compressed $120\times$, which is true real time — and its log’s first line, `E1 N=1 reps=8 00:39:40`, names the run directory the E1 artefact’s first row still points at. The arithmetic above and the flag agree.

We report that as a check rather than as the answer, because the order matters. The rate was determined from the measurements while the metadata was genuinely unavailable, and the script surfaced afterwards, by luck, from a machine that was about to be switched off. What the agreement establishes is not that E1’s provenance was fine all along — it was not — but that the recovery method returns the right answer when there is an answer to check it against. That is the one thing a physical-constraint argument can rarely demonstrate about itself.

The same script also explains a feature we had described and not accounted for. It passes `--max-t-sim 2`: two seconds of match time per run. The median of seven matched events that makes E1 under-powered, and that the main text’s attribution section treats as an opening burst, is that window. The burst is real and is why those seven events are the densest in the match; the reason there were only seven is that the campaign asked for two seconds.

We leave the disclosure standing rather than deleting it, for two reasons. The recovery was available only because the artefact we were chasing happened to be a wall-clock constant with a sharp signature, and the confirmation only because a virtual machine had not yet been destroyed; a reader should not have to reconstruct an experimental parameter by arithmetic on its results, and next time there may be nothing to reconstruct it from and nothing left to check it against. And the exercise makes the protocol rule concrete: what was missing was one number, written down once.

What the gap touched. The audit is unaffected either way: a negative transport is causally impossible at any replay rate, and the rejection counts are arithmetic on surviving per-event data. The withdrawal in the main text’s attribution section is unaffected by construction — we give the argument in a form that holds at every candidate rate. The rules of the main text’s rules section were measured after the pre-flight existed, at a derived and verified rate. the main text’s powered-transport section’s external validity is restored by the recovery above, and the recovered script now documents it as well — though the inference stood on its own before the script appeared, and is what we would still be relying on had the machine been released a day earlier. the main text’s protocol section carries the rule that would have made all of this unnecessary.

2 S2: The load-axis post-mortem, in full

Moved from the main text’s mixture development: the load-axis post-mortem in full. The main text keeps the admissions and the numbers.

Both load-axis predictions failed, including ours. The knee sweep placed conditions at achieved ρ of 0.881, 0.920, 0.950, 0.970 and 0.990, where the candidate forms diverge most; the earlier ladder stopped at 0.878 and everything above it collapsed onto a degenerate $\rho = 1.000$, which is why the main text’s rules section could not settle the question. With those points in hand, M/G/1 fits *worse than the mean* ($R^2 = -0.05$ against a fitted exponential’s 0.93). The withdrawal does not merely stand: the forms are now separated, and M/G/1 is the one ruled out.

The bounded prediction fared better but also failed. Over $\rho = 0.881$ to 0.990 the rate grew by a factor of 1.44. M/G/1 predicted 13.8; the bracket we registered before the sweep predicted 2.45–3.07. The rate saturates, as a bounded mechanism requires — but harder than our own extrapolation allowed, and a prediction that misses low is still a prediction that missed. We report it as a failure rather than as the nearer of two misses.

The two failures have one cause, and E-A5 names it. Our bracket was still parameterised in ρ , through $p = \rho^C$; E-A5 shows ρ does not determine occupancy. What survives is the mechanism — established by manipulation — and not any curve in ρ , ours included.

3 S3: The campaign ledger schema

One row per cell of every instrumented-benchmark campaign, docs/results/external_campaigns_index.csv:

- `campaign`, `cell` — sub-campaign directory and cell name; the cell name encodes the swept axis (1 load percent, `s` message size, `r` producer rate) and replicate number, e.g. `r625_rep3`.
- `level` — the numeric value of the swept axis.
- `valid` — 1 if the cell's counts are usable under the pre-stated rule; 0 rows are retained with a reason, never deleted.
- `count_source` — `shutdown_hook` for exact end-of-run counters; `periodic_quantised` for in-run lines (quantised to 10^4 , excluded from analysis); `none_in_log` when no counters were written.
- `kept`, `discarded_zero`, `discarded_negative` — the instrumented guard's three counters, separated by sign.
- `log_bytes`, `log_sha256` — size and hash of the run log the row was built from, so any row can be re-derived and checked against its source.

The Python-harness campaigns (no JVM, Section on generality in the main text) use the same cell naming and publish docs/results/external/harness_results.csv with per-cell counts, sign-separated discards, and pacer-jitter percentiles.

4 S4: OpenMessaging distributed mode, failure diagnostics

Eleven attempts in total never produced a measurement. The three diagnosed attempts of the referee campaign (chain 18, block E) fail identically at OMB commit 5b1fa70: `java.util.concurrent.CompletionException` wrapping `java.lang.IllegalArgumentException` in the coordinator-worker path, before any workload starts. Per attempt, the version, full coordinator and worker logs, and the extracted signature are archived in docs/results/external/dist_diag/; a draft upstream report is at docs/omb_distributed_issue.md (not filed — filing is the author's action).

5 S5: The E1 reconciliation, in full

Moved from the main text's powered-transport section: the complete windowed re-analysis showing E1's near-equality is the observation window and nothing else.

Why E1 saw near-equality, tested directly. The two campaigns disagree by more than power: E1 puts both systems near 0.8 ms and within 0.1 ms of each other, while the powered campaign puts them at 0.53 and 0.11 ms and 0.41 ms apart. Attributing that to seven events is an explanation only if it survives being checked, so we checked it. If the difference is the observation window and nothing else, then restricting the *powered* runs to their first seven events — in emission order, the same events E1 would have caught — must reproduce E1, and the same runs must give the powered answer over their full length. That is a prediction with one free parameter fixed in advance, and it can fail.

Table 1. The same runs reproduce both published answers. Transport medians (ms) from the powered campaign, computed over every event and over each run’s first seven events in emission order — the window E1 saw. The all-events columns reproduce the powered 0.41 ms shift; the prologue columns reproduce E1’s near-equality (the main text) in both shift and absolute level. The window, not the brokers, accounts for the disagreement.

N	Runs	All events			First 7 (prologue)		
		Kafka	Redis	Shift	Kafka	Redis	Shift
1	5	0.476	0.095	0.381	0.831	0.743	0.088
9	45	0.533	0.116	0.417	0.939	0.810	0.129
12	55	0.527	0.113	0.414	1.018	0.769	0.248

It does not fail (Table 1). Over all events the runs give shifts of 0.381, 0.417 and 0.414 ms, reproducing the powered 0.41 ms. Over their first seven they give 0.088, 0.129 and 0.248 ms — E1’s wandering near-equality, and of the same magnitude as E1’s own 0.021, 0.116 and 0.053 ms. The *absolute* medians match as well, which the shift alone does not force: the prologue puts Kafka at 0.83–1.02 ms and Redis at 0.74–0.81 ms, against E1’s 0.79–1.00 and 0.72–0.86. One set of runs, one rate, one protocol, two windows, both published answers.

So E1 is not wrong and the campaigns do not contradict each other. E1 measured the opening burst, during which both systems are slow for the reason the main text’s attribution section establishes — topic auto-creation, connection setup, and the first blocking `produce()` — and that shared start-up cost swamps a 0.41 ms difference. As the run lengthens the cost dilutes, and the brokers’ steady-state separation emerges. The seven-event corpus answered a question about start-up while appearing to answer one about brokers, which is the same failure of window selection as the main text’s attribution section, one level up.